

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2233 Programación Avanzada (2026-2)

Ejercicios propuestos 3

Interfaces gráficas y tópicos avanzados

Entrega

- **Fecha y hora límite: 23 de noviembre; 2026, hasta las 20:00 horas**.
- **Lugar:** Repositorio personal de GitHub, en la carpeta `EP3/`. El código debe encontrarse en la rama (*branch*) por defecto del repositorio: `main`.
- **Contacto:** Para informar que los entregables se encuentran disponibles y solicitar *feedback*, debes comunicarte con el/la profesor(a) de tu respectiva sección. No se aceptarán entregables enviados después de las **20:00 horas del 23 de noviembre, 2026**.
- **Condiciones de revisión:** Solo se entregará *feedback* a las entregas que cumplan con las condiciones de ejecución, formato, legibilidad y organización indicadas en la sección de Restricciones y alcances.

Objetivos

- Utilizar conceptos de interfaces y `PyQt6` para implementar una aplicación gráfica e interactiva.
- Traspasar decisiones de diseño y modelación en base a un documento de requisitos.
- Diseñar e implementar una arquitectura cliente-servidor. Además, en el cliente se debe entender y aplicar los conceptos de *back-end* y *front-end*.
- Aplicar conocimientos de señales.
- Aplicar conocimientos de creación de redes para comunicación efectiva entre computadores (*networking*).
- Aplicar conocimientos de las librerías `pandas` o `numpy` para obtener estadísticas.

Índice

1. Introducción **3**

2. *La Gran Pesca del DCCongrio* **3**
2.1. Lanzar el anzuelo 3
2.2. Atraer el pez 4
2.3. Agotar el pez 4
2.4. Capturar el pez 5

3. Arquitectura Cliente-Servidor **5**
3.1. Registro de intentos de pesca 6
3.2. Calculo de estadísticas 6

4. Parámetros **7**

5. Restricciones y alcances **7**

1. Introducción

Pensando en las vacaciones de verano que están a la vuelta de la esquina, decides cumplir el sueño de tu vida "atrapar el congrio más grande de todos los tiempos". Para prepararte para este desafió, sabes que debes mejorar tus reflejos y velocidad de reacción, por lo que decides programar un juego que te ayudará a cumplir tu sueño: *La Gran Pesca del DCCongrio*.

En este ejercicio propuesto desarrollarás el sistema completo: una **interfaz gráfica** que permita recibir *input* del usuario y mostrarle un *output*; un **servidor** que almacene información y calcula estadísticas mediante un correcto uso de las librerías `pandas` y `numpy`.

El enunciado describe qué debe hacer el programa, las responsabilidades mínimas del cliente, el servidor, y fija la estructura de archivos que permite implementar la arquitectura cliente-servidor. Cómo organizar las clases y los módulos es parte del ejercicio y queda a tu criterio.

2. *La Gran Pesca del DCCongrio*

El programa a implementar, *La Gran Pesca del DCCongrio*, consiste en un juego de pesca que consta de 4 etapas:

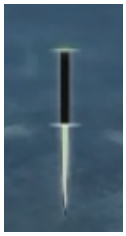
1. Lanzar el anzuelo
2. Atraer el pez
3. Agotar el pez
4. Capturar el pez

Cada etapa implicará de una interacción distinta del usuario[1] con el programa:

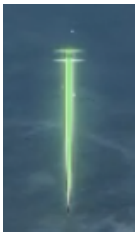
2.1. Lanzar el anzuelo

Para lanzar el anzuelo, el usuario deberá mantener apretado el *click* izquierdo del *mouse*, mientras aumenta el valor de una barra de progreso. Entré más cerca del máximo se encuentre el valor de la barra de progreso, mejor será el lanzamiento.

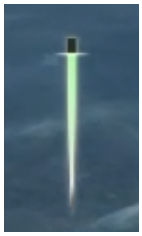
Cuando el valor de la barra de progreso llegue a su valor máximo, este decrecerá hasta llegar al mínimo y así sucesivamente.



(a) Valor en crecimiento



(b) Cerca del valor máximo



(c) Valor en decrecimiento

Figura 1: Distintos estados de la barra de progreso, en orden cronológico.

[1] A modo de simplificación, nos referiremos al "usuario" como la persona que ejecutará el programa y jugará el juego.

2.2. Atraer el pez

Para atraer el pez, el usuario deberá apretar la tecla "espacio" una cantidad definida de veces. La cantidad de veces será desconocida para el usuario, queda a tu criterio cómo se define este número.

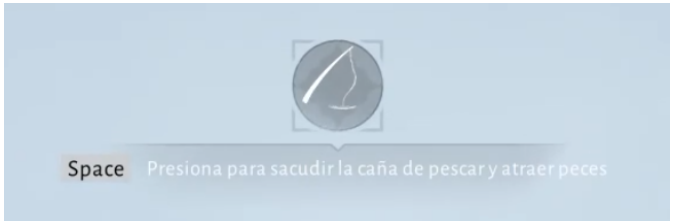


Figura 2: Interacción para atraer el pez.

2.3. Agotar el pez

Para agotar el pez, el usuario deberá ubicar un indicador hasta que se agote la estamina del pez.

El indicador se moverá en base a la posición horizontal del *mouse* con respecto al centro de la ventana. La posición del *mouse* no es la posición del indicador, sino que esta corresponde a un factor de movimiento del indicador:

- El indicador se moverá con dirección hacia la izquierda o derecha dependiendo de la ubicación del *mouse* con respecto al centro de la ventana.
- Entre mayor será la distancia del *mouse* respecto al centro, mayor será la velocidad de movimiento. Por esto, un *mouse* ubicado cerca del centro, tendrá una menor velocidad de movimiento que un *mouse* ubicado en los extremos.

Mientras el indicado se encuentre en la zona segura, la estamina del pez disminuirá; en cambio, cuando el indicador esté fuera de la zona segura, la estamina del pez se recuperará.

En esta etapa, deberás visualizar:

- **La estamina del pez:** Se representará como un anillo, el cual partirá vació. Un anillo vació implica que la estamina se encuentra en su máximo; mientras que un anillo lleno implica que se agotó la estamina del pez.
- **La posición del *mouse*:** Se representa como una barra horizontal que muestra la posición horizontal del *mouse* con respecto al centro de la ventana.
- **El indicador y la zona segura:** Se representa como un anillo donde se muestra la posición del indicador y la ubicación de la zona segura.

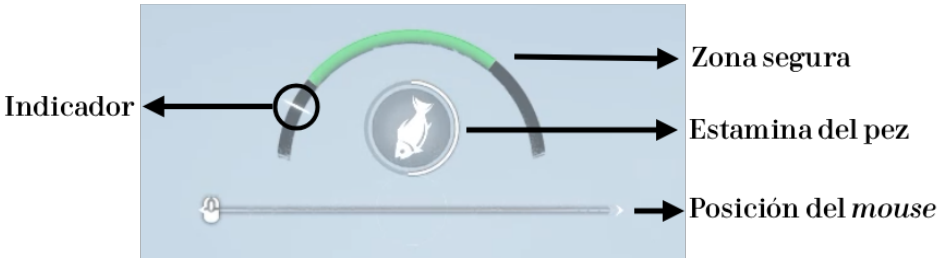


Figura 3: Distintos elementos visuales de la etapa "Agotar el pez".

2.4. Capturar el pez

Finalmente, cuando el pez se encuentre agotado, el jugador tendrá una cantidad limitada de segundos para apretar la tecla "espacio". Si lo hace, habrá capturado el pez; en caso contrario, habrá fallado la pesca.

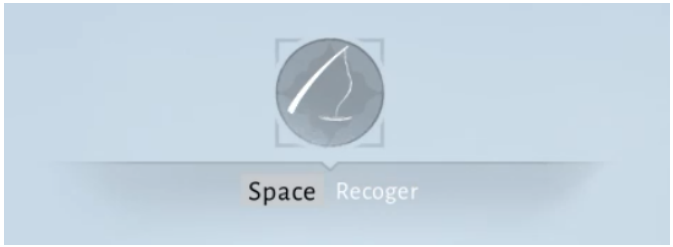


Figura 4: Interacción para capturar el pez,

Una pesca exitosa entregará un *DCCongrio*, el cual variaría su peso según la cantidad de tiempo que demoró el jugador en capturarlo y la calidad del lanzamiento del anzuelo:

$$peso_congrio = peso_mínimo + delta_peso \times e^{-\frac{tiempo_total}{60}} \times calidad_lanzamiento$$

Los valores de *peso_mínimo* y *delta_peso* quedan a tu criterio.

Todas las etapas anteriores y la interacción del usuario con el programa se encuentra resumido en el **siguiente video**. Lo que deberás implementar corresponde únicamente a las distintas barras, arcos, anillos y mensajes que muestran el estado de cada una de las etapas, correspondientes a las Figuras 1, 2, 3 y 4.

3. Arquitectura Cliente-Servidor

Además de la interfaz gráfica descrita anteriormente, tu programa deberá implementar una arquitectura Cliente-Servidor. El cliente y el servidor corresponderán a dos entidades completamente independientes que simulan encontrarse en computadores distintos. Es por ello que el primero no puede acceder a los recursos del otro, ni viceversa. En términos prácticos, el cliente y el servidor deberán estar desarrollados en directorios distintos. Cada directorio debe contener los recursos necesarios para que la entidad presente pueda ejecutarse sin dificultades, y sin acceder al directorio de otra entidad.

Un ejemplo de cómo se realiza esta separación se muestra en la Figura 5. Se puede observar que cada directorio está separado y ambos tienen su propio archivo principal `main.py` que debe ser ejecutado. Notar que estos son los únicos dos archivos que se ejecutarán.

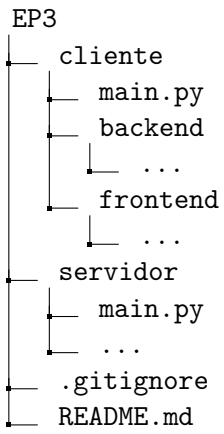


Figura 5: Organización mínima del directorio

El cliente se encargará de interactuar con el usuario, permitiendo que este: juegue *La Gran Pesca del DCCongrio*, revise las estadísticas que calculará el servidor (más información en Subsección 3.2).

El servidor se encargará de registrar todos los intentos de pesca de cada usuario y calcular estadísticas a partir de los datos registrados.

3.1. Registro de intentos de pesca

El servidor registrará tanto los intentos exitosos como los fallidos. En cada intento se deberá guardar la siguiente información:

- Identificador del jugador. Queda a tu criterio si este identificador corresponde a un nombre de usuario, un número, etc.
- Si la captura fue exitosa o no. En el caso de que sea exitosa, se deberán registrar los siguientes dos puntos:
  - Tiempo en segundos que demoró en completar la captura.
  - Peso del congrio pescado.

El formato en que se guarde la información queda a tu criterio.

3.2. Calculo de estadísticas

A partir de los datos almacenados en el servidor, deberás obtener las siguientes estadísticas:

a. Estadísticas generales de todos los datos registrados:
   - Porcentaje de capturas exitosas.

b. Estadísticas de un usuario en específico:
   - Cantidad de capturas exitosas.
   - Porcentaje de capturas exitosas.
   - Tiempo promedio y menor tiempo de captura.
   - Peso promedio y peso mayor de los *DCCongrio* capturados.

4. Parámetros

Todos los valores que correspondan a parámetros como nombres de archivos y valores constantes deben definirse en el archivo `parametros.py`. Tanto el cliente como el servidor, tienen su propio archivo de parámetros.

5. Restricciones y alcances

- Este ejercicio es **estrictamente individual** y está regido por el **Código de Honor** de Ingeniería.
- Tu programa debe ser desarrollado utilizando Python **3.12.X**, con X mayor o igual a 0.
- Tu programa debe estar compuesto por uno o más archivos con extensión `.py`, organizados coherentemente dentro de la carpeta de entrega. **No se revisarán archivos en otros formatos**, como `.ipynb`.
- El programa será ejecutado **únicamente desde la terminal**, ubicando la terminal dentro de `servidor/` o de `cliente/` según corresponda y utilizando el siguiente comando:

  ```
  python3 main.py
  ```

- Durante la revisión se podrá cambiar el nombre de la carpeta que contiene el **Ejercicio Propuesto** por otro nombre. Por lo tanto, los *paths* relativos utilizados en el programa deben ser coherentes.
- Solo se revisarán entregables que puedan ejecutarse correctamente desde la consola. Si el programa no puede iniciarse, se cierra inesperadamente antes de acceder a sus principales funcionalidades o requiere pasos no documentados, el entregable podrá no ser revisado y no se entregará *feedback*.
- La consola del cliente debe ser robusta frente a opciones inválidas, texto en campos numéricos e identificadores inexistentes; el servidor debe ser robusto frente a desconexiones y mensajes corruptos. Se permite utilizar `try/except` para validar conversiones, operaciones de archivos y de red, **especificando siempre la excepción que se captura**. No se acepta el uso de "`except:`" ni "`except Exception:`".
- Se espera que el código sea legible, esté organizado coherentemente y cumpla con las buenas prácticas de programación trabajadas en el curso. En particular, se espera:
  - Utilizar nombres descriptivos para variables, funciones, clases y módulos.
  - Distribuir las responsabilidades del programa de manera coherente.
  - Evitar la repetición innecesaria de código.
  - Evitar concentrar toda la solución en un único archivo, función o componente; ningún archivo debería superar las 400 líneas.
  - Mantener un formato consistente que facilite la lectura del código (PEP8, líneas de hasta 100 caracteres).
  - Documentar las decisiones que no sean evidentes.

  Si el código no presenta condiciones mínimas de legibilidad u organización que permitan comprender su funcionamiento, el entregable podrá no ser revisado y no se entregará *feedback*.
- Debe existir una separación clara *front-end*/*back-end* y cliente/servidor. Puedes distribuir el código en los módulos que consideres apropiados, respetando los nombres obligatorios de la Sección 3.

- Solo se permiten módulos de las biblioteca estándar de Python (en particular `socket`, `threading`, `random`, `json`, `pickle`, `datetime`, `math`, `os`, `sys` y `time`) más `PyQt6`, `flask` y `requests`. Si no se encuentra especificado en el enunciado, supón su uso **está prohibido**. Pregunta en una *discussion* en el **foro** si es que es posible utilizar alguna librería en particular.
- Este ejercicio debe desarrollarse **exclusivamente con los contenidos liberados** como materia regular al momento de publicar el enunciado.
- Está prohibido citar o utilizar código publicado **después de la liberación del enunciado**. Solo se permite citar contenido que ya se encontrara disponible antes de su publicación, referenciándolo en el `README.md`.
- Ante cualquier duda relacionada con el enunciado, puedes utilizar ***Discussions***.
- Cualquier aspecto no especificado queda a tu criterio, siempre que no contradiga una regla explícita del enunciado. Los supuestos realizados deben aplicarse consistentemente y quedar explicados en el archivo `README.md`, junto con las instrucciones de ejecución, el estado de la entrega.

Los entregables que no cumplan con las restricciones de ejecución, formato o legibilidad podrán no ser revisados y no recibir *feedback*.